

Jane Street Reverse Engineering an ASIC

By Paul Vincent Bezzina

Contents

1	Tools used	2
2	Extracting the netlists	3
2.1	Creating the Verilog Netlist file	3
3	Creating each module	4
3.1	The List	4
3.2	RTL Overview	6
4	Taking a look at the test case example	7
5	Running a test bench	8
5.1	Example code	8
6	Grouping things together	11
6.1	Comments	11
6.2	Spatial Proximity using the geometric data in the GDS file	11
6.3	Removing dead weight	12
6.4	The Clusters we now have	12
6.4.1	Cluster 4 and Cluster 2	12
6.5	Testing the clusters	13
6.6	The simplification process	14
7	Cracking the Success Condition	15
7.1	Revisiting the “no brute force” rule	15
7.2	Why a SAT solver	15
7.3	The boolean formula	16
7.4	The 121 cycle assumption was off by one	17
7.5	A ghost in the netlist	17
8	The winning sequence	18
8.1	Reflections	18
8.2	Verification	18
9	Easter Eggs	19
9.1	The date	19
9.2	The test case outputs	19
9.3	TWO STARS	19

1 Tools used

- Quartus Prime
- Verilog
- ModelSim
- Python scripts
- SAT
- The Warm Up Puzzle as a reference

2 Extracting the netlists

2.1 Creating the Verilog Netlist file

Using a netlist extractor python file I was able to create a Verilog file with all the netlists and instantiated modules prepared. The only issue with this file is that the modules at the moment have no logic behind them; thus if we were to attempt to test this Verilog file nothing would happen. This will be a challenge left for me to solve. Furthermore, there is no logic behind the list of connections; they are not grouped.

For anyone curious how a tool like this actually works: a GDS file only contains geometry, polygons on named layers, cell instance placements with x and y coordinates and an orientation, and nothing that looks like a logical netlist at all. Getting a structural Verilog file out of that means walking every placed instance, matching its footprint against the sky130_fd_sc_hd standard cell library to work out which cell it actually is, then tracing the metal and via layers between every cell's pins to figure out what connects to what. Once that connectivity graph exists, writing it back out as a list of module instantiations with named nets, which is exactly what the snippet below is, is the easy part.

An example snippet:

```
1 sky130_fd_sc_hd__o21a_2 inst_1 (  
2     .VGND(VGND),  
3     .VPWR(VPWR),  
4     .A2(n__1),  
5     .A1(n__23),  
6     .B1(n__16),  
7     .X(n__104)  
8 );  
9 sky130_fd_sc_hd__and2_2 inst_7 (  
10     .A(n__103),  
11     .VGND(VGND),  
12     .X(n__1),  
13     .VPWR(VPWR),  
14     .B(n__21)  
15 );  
16 sky130_fd_sc_hd__dfxtp_2 inst_9 (  
17     .Q(n__1351),  
18     .VPWR(VPWR),  
19     .VGND(VGND),  
20     .CLK(n__3591),  
21     .D(n__3923)  
22 );
```

3 Creating each module

First of all, I will try to create a list of all the unique modules in this netlist so that I can more effectively begin to create these modules. I have opted to try and create these modules myself rather than AI generating them simply because I want to improve my Verilog skills.

3.1 The List

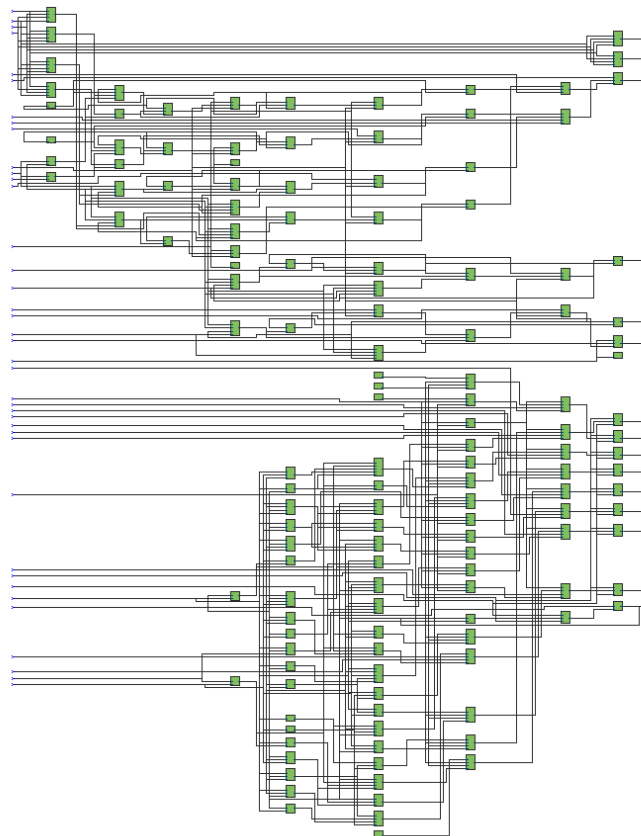
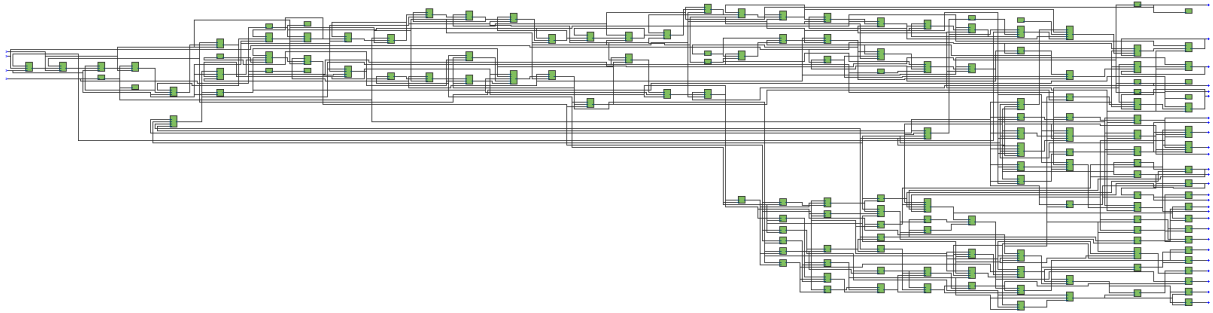
Count	Cell	Function
84	dfirtp_2	D flip flop, active LOW async reset
49	nor2_2	2 input NOR
39	nand2_2	2 input NAND
31	o21a_2	Complex gate (OR AND, grouping '21')
30	and2b_2	2 input AND (1 inverted input)
29	xnor2_2	2 input XNOR
26	a31o_2	Complex gate (AND OR, grouping '31')
26	and3_2	3 input AND
25	inv_2	Inverter
24	nand2b_2	2 input NAND (1 inverted input)
23	a22o_2	Complex gate (AND OR, grouping '22')
21	xor2_2	2 input XOR
21	mux2_1	2:1 multiplexer
20	a21oi_2	Complex gate (AND OR INVERT, '21')
19	a21o_2	Complex gate (AND OR, '21')
18	or3_2	3 input OR
17	and2_2	2 input AND
16	clkbuf_8	Clock buffer
15	clkbuf_4	Clock buffer
15	nand4_2	4 input NAND
14	and4bb_2	4 input AND (2 inverted inputs)
13	or2_2	2 input OR
12	o211a_2	Complex gate (OR AND, '211')
11	o31a_2	Complex gate (OR AND, '31')
10	or4_2	4 input OR
9	or4b_2	4 input OR (1 inverted input)
8	and4_2	4 input AND
6	conb_1	Constant tie hi/tie lo
6	o21ai_2	Complex gate (OR AND INVERT, '21')
6	a221o_2	Complex gate (AND OR, '221')

Continued on next page

Count	Cell	Function
5	nor3b_2	3 input NOR (1 inverted input)
5	a211o_2	Complex gate (AND OR, '211')
5	a32o_2	Complex gate (AND OR, '32')
4	dfxtp_2	DFF (no reset)
4	dfstp_2	DFF (async set)
4	a21boi_2	Complex gate
4	and4b_2	Complex gate
4	and3b_2	Complex gate
4	nor3_2	Complex gate
4	o32a_2	Complex gate
4	o22a_2	Complex gate
3	a211oi_2	Complex gate
3	o221a_2	Complex gate
2	nand3_2	Complex/basic gate
2	nor4_2	Complex/basic gate
2	nor4b_2	Complex/basic gate
2	o22ai_2	Complex/basic gate
2	o311a_2	Complex/basic gate
2	o21ba_2	Complex/basic gate
2	o31ai_2	Complex/basic gate
2	a21bo_2	Complex/basic gate
2	a311o_2	Complex/basic gate
1	or3b_2	Various complex/basic gate
1	a221oi_2	Various complex/basic gate
1	a31oi_2	Various complex/basic gate
1	a41oi_2	Various complex/basic gate
1	nand3b_2	Various complex/basic gate
1	or4bb_2	Various complex/basic gate
1	o32ai_2	Various complex/basic gate
1	a22oi_2	Various complex/basic gate
1	o2bb2a_2	Various complex/basic gate
1	a2111oi_2	Various complex/basic gate
1	buf_2	Various complex/basic gate
1	clkbuf_16	Various complex/basic gate
1	o211ai_2	Various complex/basic gate
1	o21bai_2	Various complex/basic gate

The list was formatted based on the number of times each one of these gates appeared.

3.2 RTL Overview



4 Taking a look at the test case example

Despite the inputs for both cases being different, the output remains the same. The output also spells out TRY AGN, assuming this means Try Again; we could make a hypothesis here that this circuit will either output TRY AGN, or the secret string, with no in between. The question though still remains at this point: what input does the circuit want exactly?

Despite these many new questions, at least we now know that the output generator would seem to just be an ASCII character generator and is pointless for now to observe, as the blog suggested.

Important to note that the circuit only works when RST is HIGH, and only begins outputting data once EN is low. Additionally, the circuit will not work if the number of pulses is less than 121.

5 Running a test bench

With the netlist Verilog file completed I was able to then create each module in a separate file that was instantiated, and thus I was able to actually successfully reconstruct this circuit in Quartus Prime. Using the testbench given and converting it into Verilog, I can confirm using ModelSim that we actually got the exact same output as in the example test.

I did consider brute forcing the ASIC to get the success combination and from there reverse engineer the ASIC; however I felt this went against the spirit of the puzzle and thus my next course of action will be to go back to the ASIC and begin deducing what each section does and extract the correct combination.

Note to self in this section: the number of clock cycles required is 121 clock pulses. This code does not work at all if the number of clock pulses is less. (Revisited later, in Section 7; this turned out to be the correct floor but not, as I assumed at the time, also the ceiling.)

5.1 Example code

```
1  `timescale 1ps/1ps
2  module testbench_try_solution;
3      parameter CLK_PERIOD = 10000;
4      parameter ENABLE_CYCLES = 121;
5      parameter SETTLE_CYCLES = 5;
6      reg clk, rst_n, enable, I;
7      wire O_0_, O_1_, O_2_, O_3_, O_4_, O_5_, O_6_, O_7_, success;
8      wire [7:0] output_hex = {O_7_, O_6_, O_5_, O_4_, O_3_, O_2_,
9                               O_1_, O_0_};
10     reg success_seen;
11     integer success_time;
12     integer i;
13     reg candidate [0:120];
14     initial begin
15         for (i = 0; i <= 120; i = i + 1)
16             candidate[i] = 1'b0;
17     end
18     JaneStreet dut (
19         .I (I),
20         .clk (clk),
21         .enable (enable),
22         .rst_n (rst_n),
23         .O_0_ (O_0_), .O_1_ (O_1_), .O_2_ (O_2_), .O_3_ (O_3_),
24         .O_4_ (O_4_), .O_5_ (O_5_), .O_6_ (O_6_), .O_7_ (O_7_),
25         .success (success)
```

```

25 );
26 initial clk = 0;
27 always #(CLK_PERIOD/2) clk = ~clk;
28 initial begin
29     $dumpfile("try_solution.vcd");
30     $dumpvars(0, testbench_try_solution);
31 end
32 always @(posedge clk) begin
33     $display("t=%0t_rst_n=%b_enable=%b_I=%b->_0=0x%02h_
34             success=%b",
35             $time, rst_n, enable, I, output_hex, success);
36     if (success === 1'b1 && !success_seen) begin
37         success_seen = 1'b1;
38         success_time = $time;
39         $display(">>>_SUCCESS_asserted_at_t=%0t_<<<", $time);
40     end
41 end
42 initial begin
43     success_seen = 1'b0;
44     // Reset
45     rst_n = 1'b0;
46     enable = 1'b0;
47     I = 1'b0;
48     repeat (3) @(negedge clk);
49     rst_n = 1'b1;
50     @(negedge clk);
51     enable = 1'b1;
52     for (i = 0; i < ENABLE_CYCLES; i = i + 1) begin
53         @(negedge clk);
54         I = candidate[i];
55     end
56
57     enable = 1'b0;
58     repeat (SETTLE_CYCLES + 20) @(negedge clk);
59     $display("===_Run_complete_===");
60     $display("Final_output_hex=_0x%02h", output_hex);
61     if (success_seen)
62         $display(">>>_RESULT:_success_went_HIGH_at_t=%0d_<<<"
63                 , success_time);
64     else
65         $display(">>>_RESULT:_success_never_asserted_during_
66                 this_run_<<<");
67
68     $finish;
69 end
70 initial begin

```

```
67         #((ENABLE_CYCLES + SETTLE_CYCLES + 20) * CLK_PERIOD);
68         $display("TIMEOUT--run did not finish in time");
69         $finish;
70     end
71 endmodule
```

6 Grouping things together

6.1 Comments

An interesting parallel here between this larger puzzle and the warmup we were given is that both take a single input and output 8 bits. This suggests that there is a shift register present in this circuit. Thus we will try to identify it by identifying the shift register on the warmup circuit and then identifying an almost identical logic block on the puzzle circuit. If a shift register is not present, this would suggest that the entire string of bits is analysed at once rather than in 8 bit segments.

Additionally, it would be logical to assume that our success output depends on a comparator which is just constantly comparing the inputs of our circuit with the ideal scenario. Unfortunately just identifying this functional block and then grouping up the modules in our netlist based on what it is will not be enough, since we need to also be able to deduce what the success criteria is. Do keep in mind though that this comparator hypothesis does not hold up that well since there are no characteristic comparator logic blocks next to the success pin, so do be open to an alternative idea.

Something else that seems a bit odd from our testbench is that this circuit seemingly only works if EN is on for 121 clock pulses and will output 0 if the clock pulses are below 121. Either all 121 bits or more inputted are used for analysis, or perhaps there is a register saving an 8 bit window of bits from the shift register every 8 clock cycles, and somehow these are compared.

Being now in possession of an RTL layout of the ASIC that works will make it easier to analyze and assess what is going on.

6.2 Spatial Proximity using the geometric data in the GDS file

Beyond geometry and connectivity, the GDS file also carries every cell's physical x and y coordinate on the die. The puzzle has been kind enough to put blocks of logic close together in proximity. That makes coordinate proximity a reasonable way to simplify the design logically. Two cells a place and route tool pushed next to each other are more likely, though by no means guaranteed, to belong to the same piece of logic than two cells sitting on opposite corners of the die.

This is how the 19 spatial clusters referenced throughout the rest of this document were first formed, before any of the logical tracing described later. A python script was used to deduce the proximity and create the clusters with a threshold of 5.5. It is worth being upfront that this is an approximation. Proximity clustering is what got the netlist down to something the size I could actually look at. One genuine bug found much later on, the `n__1495` collision documented in Section 7, traces directly back to this step: two unrelated signals that only happened to be numbered alike ended up split across the same physical boundary, and the tool doing the splitting lost track of one of them in the process.

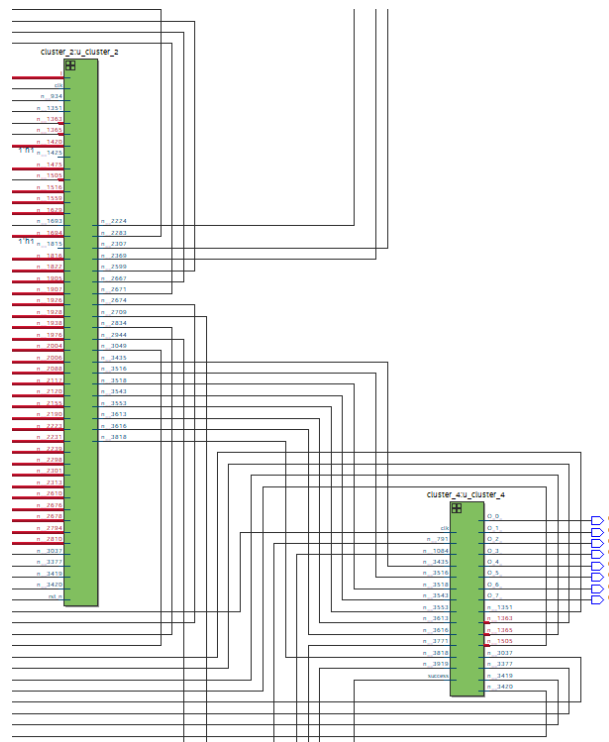
6.3 Removing dead weight

There are a lot of things in the ASIC that are relevant for dealing with real world problems such as impedance etc.. However, we can remove a lot of this for our simulation. The clock buffers only stand to add confusion to us when viewing the RTL of our synthesized circuit, thus I opted to remove them all.

Any individual not gates were simplified into a not input on the gate it led to.

6.4 The Clusters we now have

6.4.1 Cluster 4 and Cluster 2



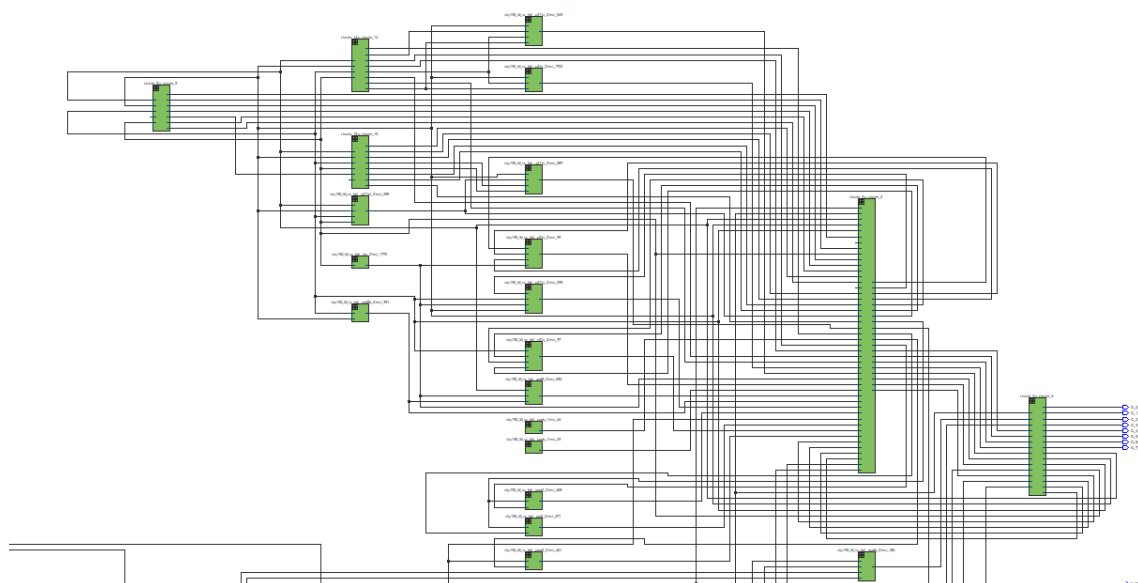
Upon investigation and testing of these two clusters it would seem to be evident that these determine the ASCII characters that are generated. Interestingly upon trying a test case where the success pin was forced to high, nothing happened. There was one strange case where forcing success HIGH on the 121st clock cycle and then turning it off on the next clock cycle caused an output of TRY AGN.

6.5 Testing the clusters

[illegible]

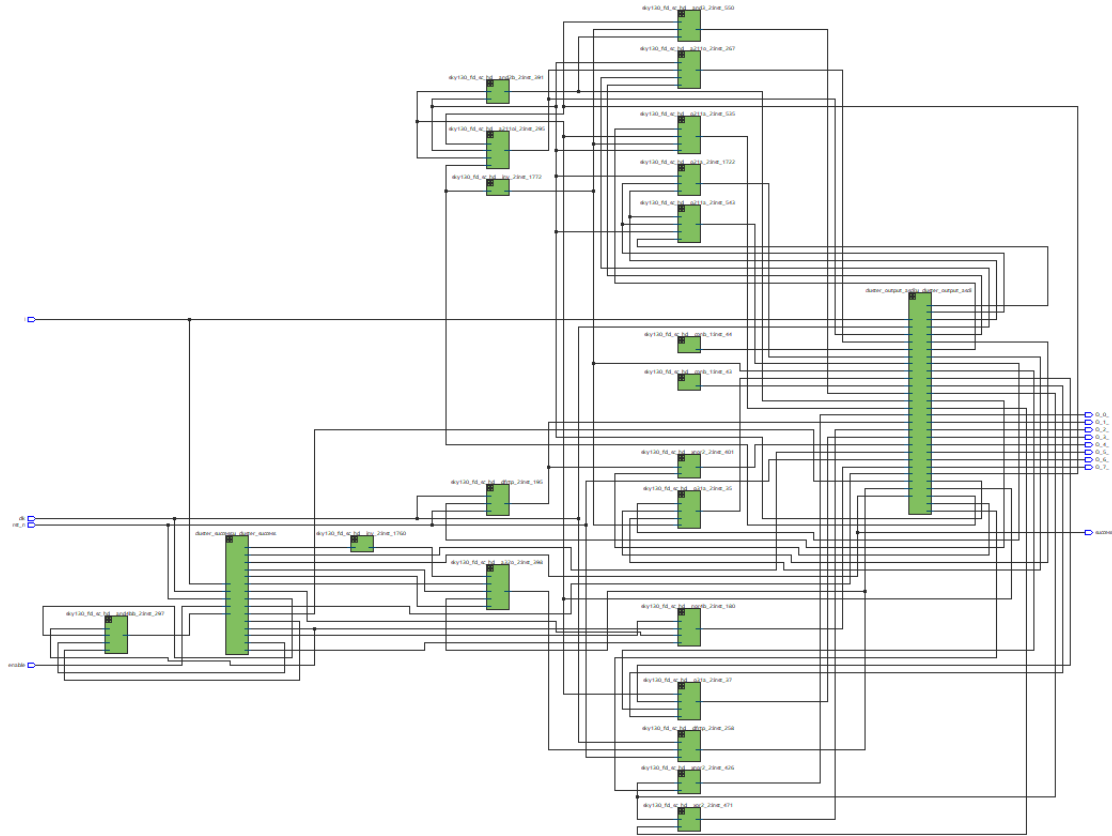
An example of us running a series of inputs into a cluster and getting its output. From this hopefully we can deduce what the function of this cluster is.

Through various other tests that have not been mentioned here, we did many isolated tests on clusters 2, 4, 5, 14 and 15 which on the RTL logic seem quite disconnected from the rest of the circuit besides the success pin. Thus I have concluded these clusters are actually to do with the ASCII generator entirely and thus I have put these into a bigger block.



6.6 The simplification process

So the ASCII character generator block was put into one block we can now just ignore, and we have also put the rest of the logic into another block. I will now also put all the logic in between into another block, the more this easier for the brain to look at the more progress we can make.



With the design now reduced to three clean blocks; the ASCII generator, the success relevant logic, and the small amount of glue sitting between them; I finally had something I could actually reason about, rather than 694 gates of undifferentiated soup. The obvious next question was: does this reduction actually tell us anything about what the correct input is, or have I just made the same problem easier to look at?

7 Cracking the Success Condition

7.1 Revisiting the “no brute force” rule

I said earlier that I didn’t want to brute force the success combination directly, since guessing at random across a 2^{121} search space felt like it defeated the point of the exercise. By this stage, though, the situation had genuinely changed. The block I was calling the success block wasn’t an opaque 694 gate netlist any more; it was a fully isolated, boundary verified circuit with exactly four real external inputs (`I`, `clk`, `enable`, `rst_n`) and one internal feedback wire (`n_1029`), and I had already proven that feedback wire depends on nothing but the block’s own outputs. Every gate inside it had been traced, and every gate outside it had been shown to be irrelevant to `success` entirely.

Searching *within* that specific, already understood block, using a method that is grounded in the real gate level logic rather than throwing darts at the whole chip, felt like a different exercise to me than the brute force I’d ruled out earlier. It’s closer to solving an equation once you’ve actually derived it, than to guessing the answer outright. So I was comfortable bringing in a solver here, on the condition that the reduction down to “here is exactly what needs to be satisfied” had been earned through the reverse engineering already done, not skipped past.

7.2 Why a SAT solver

Every deterministic and random 121 bit sequence I had tried up to this point; all zeros, all ones, alternating, several thousand random draws; came back with `success` stuck at 0. Given how much of the chip’s fan in actually feeds `success` (I traced this properly: it turned out to be roughly two thirds of the entire design, and notably *none* of clusters 2, 4, 5, 14 or 15; i.e. none of the ASCII generator), a signal with that much internal state behind it, which also self holds once triggered (it feeds back into its own trigger gate), has exactly the shape of a large, obfuscated comparator. Some accumulated piece of internal state is being compared against one specific target, and every other input falls through to a default. That is precisely the kind of problem SAT solvers are built for, even when the raw search space is far too big to brute force.

The approach: take the isolated success block’s gate list, unroll it across N clock cycles (each cycle gets a fresh free boolean variable for `I`, and every flip flop’s value simply carries forward from the previous cycle’s already computed `D`, since that is literally what $Q(t+1) = D(t)$ means), encode every gate as CNF via a standard Tseitin transformation, and ask the solver whether there is any assignment to those `I` variables that makes `success` = 1 at some point. If satisfiable, the solver hands back a working input sequence directly.

7.3 The boolean formula

It is worth actually writing out what gets solved rather than just describing it. **success** itself is just the registered output of one specific gate, an **a32o_2** cell (**inst_397**), sitting directly in front of its own flip flop. Written as a boolean expression instead of a schematic, that gate computes:

$$D_{success} = (n_{2258} \wedge n_{4166} \wedge n_{4201}) \vee (success \wedge n_{4236}) \quad (1)$$

and since **success** is nothing more than the registered value of that expression one clock later:

$$success(t+1) = D_{success}(t) \quad (2)$$

The part that actually matters is that n_{2258} , n_{4166} , n_{4201} and n_{4236} are not primary inputs. They are themselves the outputs of a few hundred gates of upstream combinational logic, each of which is in turn some boolean function of the input bit at that cycle, $I(t)$, and whatever state got carried over in the flip flops from the cycle before. So $success(t+1)$ genuinely is a large, fully expanded boolean function of every input bit up to and including $I(t)$, and asking whether the ASIC can ever be made to assert success is really asking whether there exists some assignment to $I(0), I(1), \dots, I(121)$ such that:

$$\exists I(0), I(1), \dots, I(121) \in \{0, 1\} : success(122) = 1 \quad (3)$$

Every gate in the cone gets turned into a small set of clauses over fresh boolean variables using a Tseitin transformation, so a plain 2 input AND gate with inputs a , b and output z becomes the CNF encoding of $z \Leftrightarrow (a \wedge b)$:

$$(\neg z \vee a) \wedge (\neg z \vee b) \wedge (z \vee \neg a \vee \neg b) \quad (4)$$

and a 2 input OR gate becomes the equivalent encoding of $z \Leftrightarrow (a \vee b)$:

$$(z \vee \neg a) \wedge (z \vee \neg b) \wedge (\neg z \vee a \vee b) \quad (5)$$

Every other gate type in the design, the AND OR INVERT and OR AND INVERT cells especially, reduces to a short chain of exactly these two building blocks. Chaining every gate in the success cone through this transformation, once per clock cycle across all 122 cycles, produces one CNF formula with roughly 66 thousand variables and a few hundred thousand clauses. The satisfying assignment the solver hands back for it, read off at just the $I(t)$ variables, is the winning 122 bit input sequence. And we finally solved it!

7.4 The 121 cycle assumption was off by one

My working assumption, based on the testbench behaviour noted back in Section 5, was that 121 pulses was both the floor *and* the ceiling; that if a solution existed, it had to exist within exactly 121 cycles. Constraining the solver to check `success = 1` at exactly cycle 121 came back UNSAT immediately. Relaxing the constraint to “`success = 1` at any point within a 200 cycle horizon” instead came back SAT, with the earliest possible trigger at cycle index 121; i.e. after **122** bits of `I`, not 121.

The “at least 121 pulses” rule from earlier was correct. It just wasn’t also the correct upper bound. The real minimum length is one bit longer than every test I had run.

7.5 A ghost in the netlist

Simulating the 122 bit candidate against the real gate level cell logic produced something odd: `success` didn’t resolve to 0 or 1, it went to `X`. Tracing backwards, one of the signals feeding directly into `success`’s own trigger gate, `n__4166`, was permanently undefined for the entire simulation, regardless of input. Following it one step further: `n__4166` depends on a net called `n__1495`; and `n__1495` turned out to be two entirely different, unrelated signals that just happen to share a name. One is a private wire that lives entirely inside what I had grouped as Cluster 9. The other is a reference made by a gate sitting outside every named cluster, at the very top level, to a net that nothing at that scope ever actually declares or drives.

I confirmed the exact mechanism by cross checking against the flat, non hierarchical netlist; the one extracted directly from the GDS, before it had been split into the 19 spatial clusters at all. In that file the picture is completely unambiguous: an inverter computes `n__1557 = ~n__1495` from Cluster 9’s local flip flop, and exactly one downstream gate consumes `n__1557`. Since that inverter has only the one consumer, folding its inversion directly into that consumer as a pre inverted input pin (turning a plain `AND` into an `AND` with one inverted input) is a completely standard, logically identical optimisation; it just removes a redundant gate. The tool that performed the clustering step made that substitution correctly, but never promoted `n__1495` itself to an output port of Cluster 9 so the now directly referencing gate could actually reach it across the module boundary that had been introduced.

8 The winning sequence

With that fixed, the same 122 bit candidate resolves cleanly: **success** asserts and self holds for the rest of the run, and in place of TRY AGN, the ASCII generator spells out:

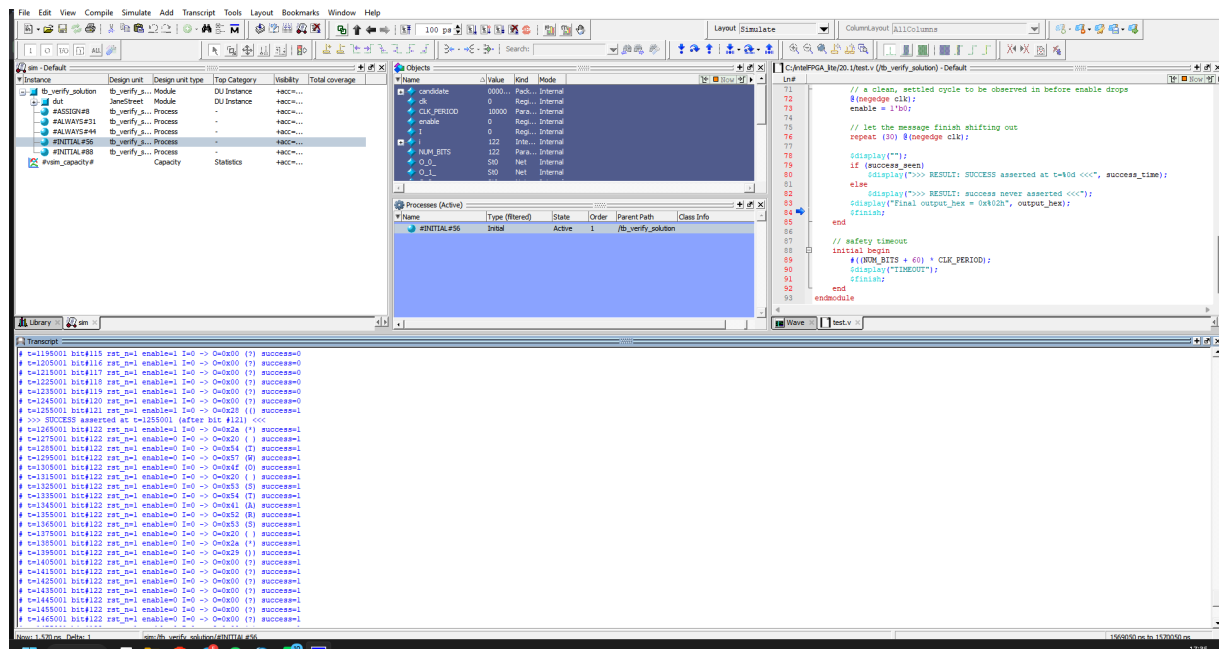
(* TWO STARS *)

```
1 // Winning 122 bit I sequence (bit 0 applied first)
2 0000000101010000100000000000010101010000000000001010000001000
3 0010000001000001010000100000001000000100000100100010100000000
```

8.1 Reflections

Looking back, the two things that actually cracked this were both direct products of the earlier, slower work rather than shortcuts around it: the boundary tracing that isolated exactly which gates **success** depends on is what made the SAT encoding small and tractable rather than a search over the whole chip, and it was only because I already understood cluster boundaries in detail that the `n__1495` collision was even findable, rather than just another unexplained X in a waveform.

8.2 Verification



Although hard to see with this image resolution, we got an output of (* TWO STAR *).

9 Easter Eggs

9.1 The date

Sat Dec 31 23:59:60 2016 was the moment a leap second was added to Coordinated Universal Time (UTC) to synchronize atomic clocks with earth’s slowing rotation.

9.2 The test case outputs

The test case output spells out TRY AGN in ASCII, and it may be safe to assume this is short for Try Again. Potentially ”EMPTY SKY” may be an easter egg here; it is a unique output that arises when all the inputs are 0. Empty Sky is also the name of an Elton John song. When the inputs are all 1, the output is BIG BANG.

9.3 TWO STARS

The actual answer, (`* TWO STARS *`), is worth a second look purely as a piece of formatting: the (`* ...*`) are exactly block comment syntax in several languages (OCaml and Pascal among them), which feels like a deliberate touch rather than an accident; the ASCII is, after all, “commenting” on itself once you actually get it to agree with you. I don’t have a confident read on what “two stars” itself is referring to; I’m noting it here rather than asserting a meaning I can’t back up, in the same spirit as the EMPTY SKY / BIG BANG entries above.